



OpenSCE Middleware and Tools set for Cluster and Grid System

Putchong Uthayopas

Director of High Performance Computing and Networking Center
Associate Professor in Computer Engineering
Faculty of Engineering, Kasetsart University
Bangkok, Thailand



OpenSCE : Scalable Cluster Environment



- An open source project that intends to deliver an integrated open source cluster environment
- Phase 1: 1997-2000 as a SMILE project
 - Scalable Multicomputer Implemented using Lowcost Equipment
- Phase 2: 2001-2003 OpenSCE project
- www.opensce.org



SCE Components

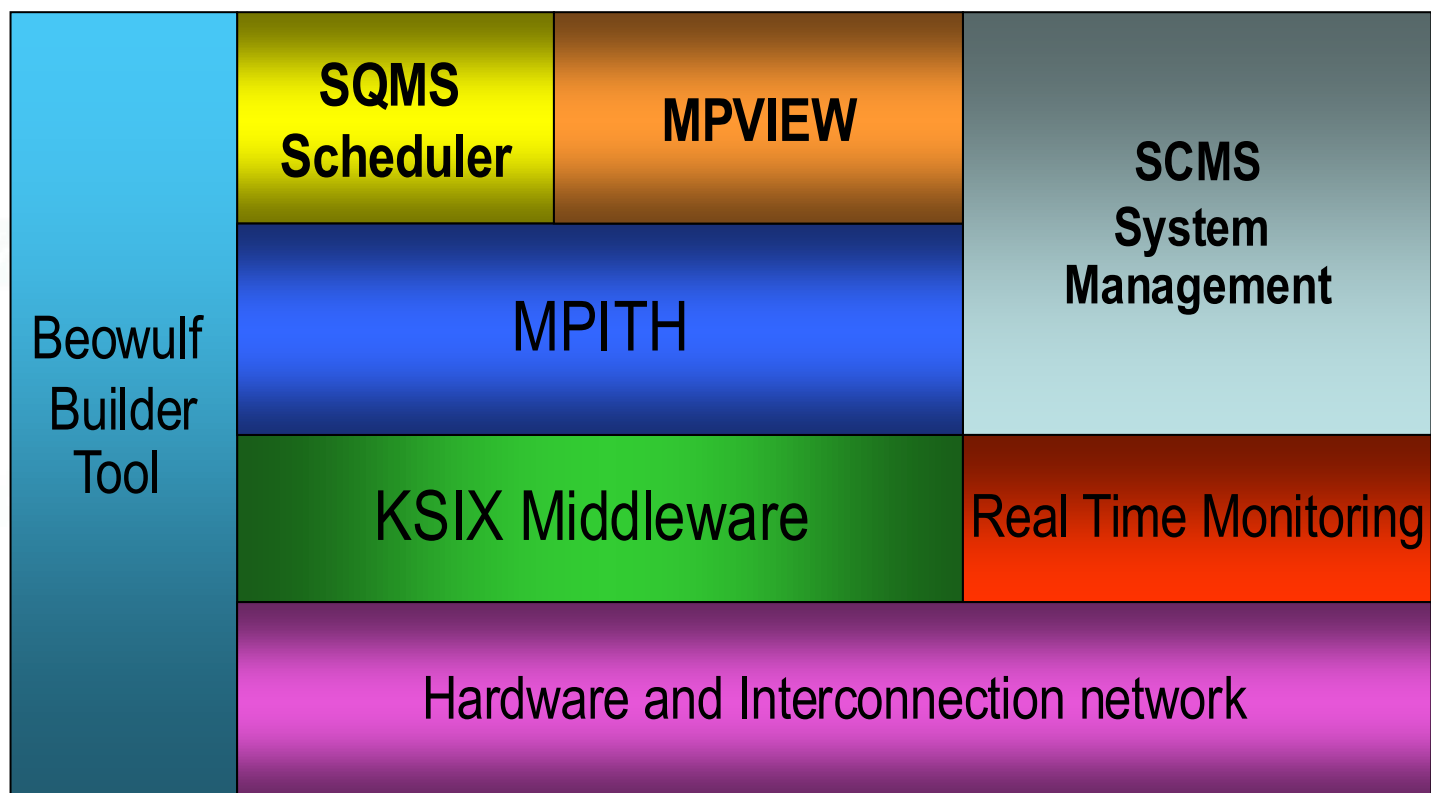


MPview – MPI program visualization

- MPITH – Quick and simple MPI runtime
- SQMS – Batch scheduler for cluster
- SCMS/ SCMSWEB cluster management tool
- Beowulf Builder (BB, SBB) cluster builder
- KSIX – cluster middleware



SCE Structures





KSIX Middleware



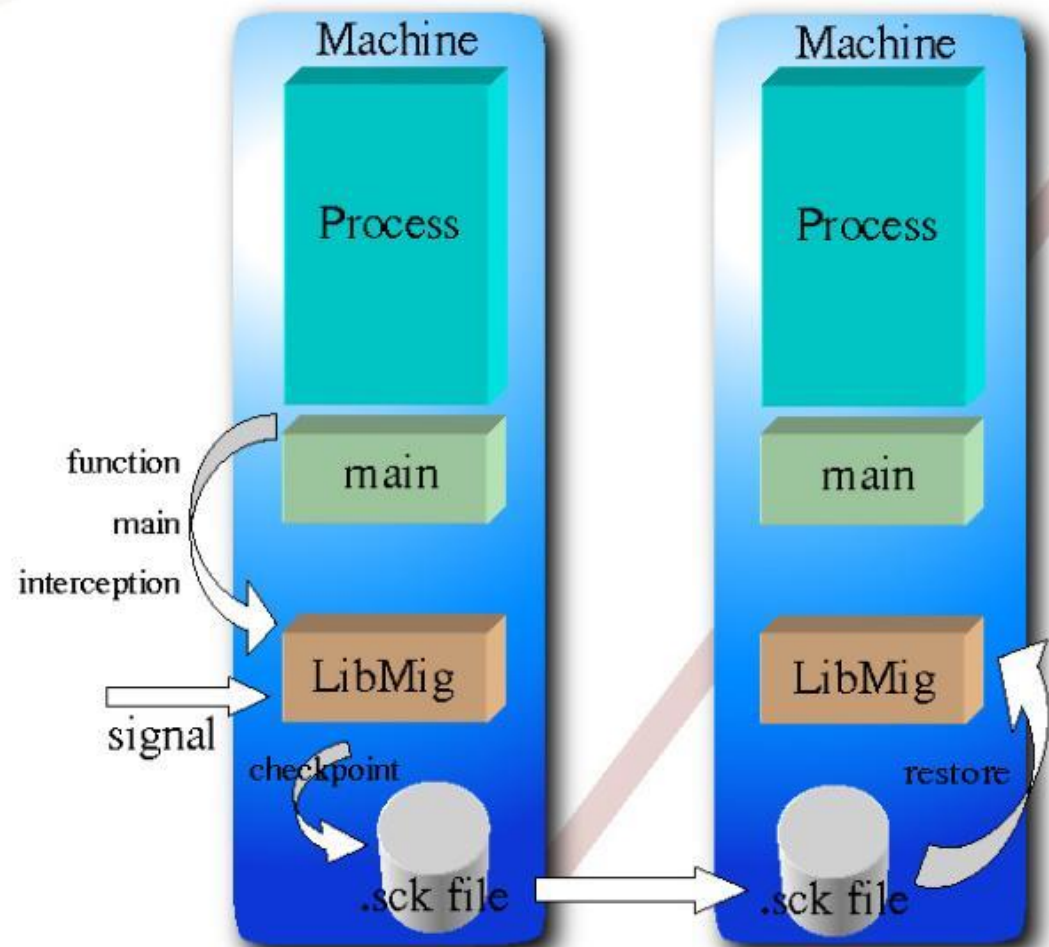
- Presenting a single system image to application
 - Unify process space, process group
 - Distributed signal management
 - Membership services
 - Simple I/O redirection



KSIX User Level Process Migration



- LibMIG
 - Checkpointing
 - Migration
 - Pure user level code
 - No recompilation
- Next version of KSIX will support load balancing
- Algorithm?



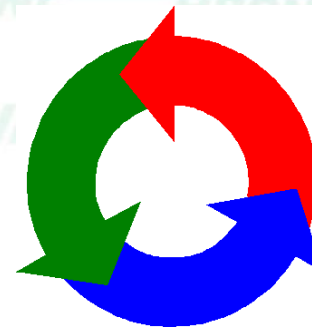
แผนภาพแสดงขั้นตอนการอพยพงาน



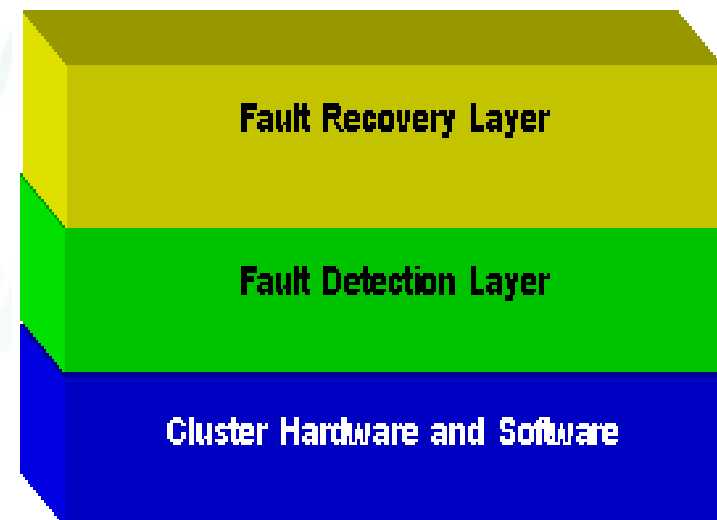
AMATA HA architecture



- AMATA is a project to build
 - scalable high availability extension to linux clustering
- AMATA
 - Define uniform HA architecture on Linux
 - Services, API, Signal



AMATA

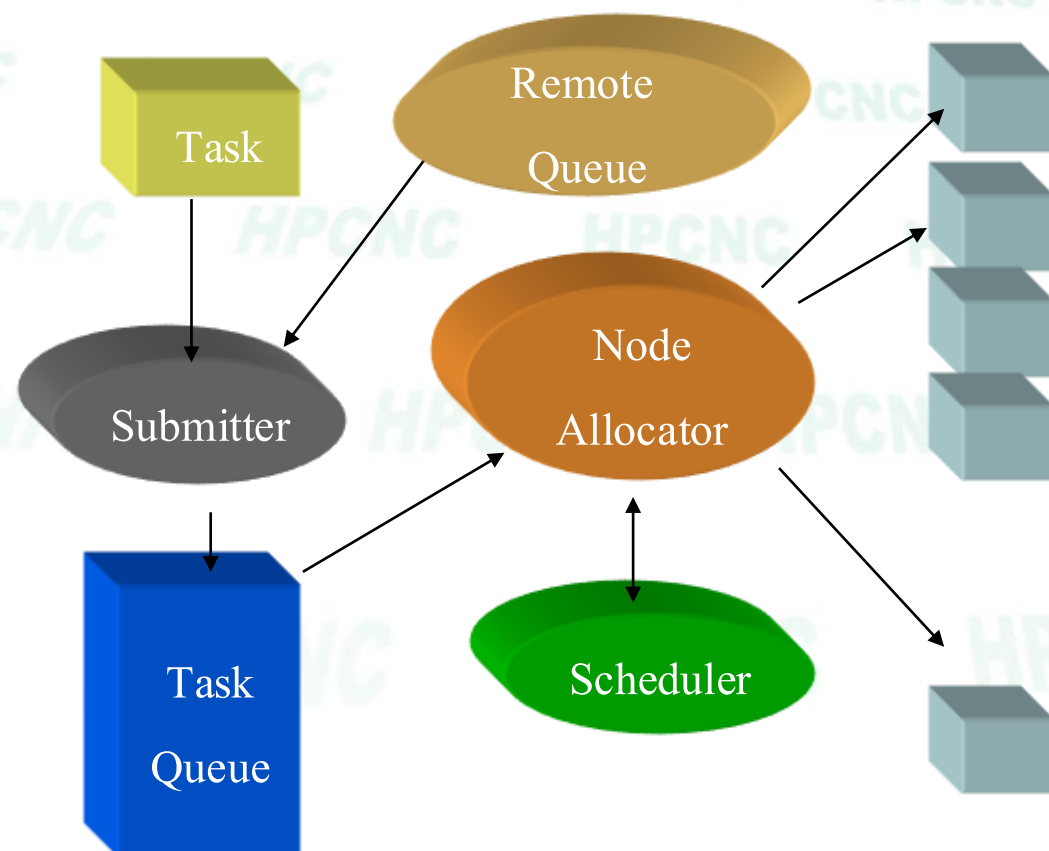




SQMS: Queuing Management System



- Batch scheduler for sequential and parallel MPI task
- Static and dynamic load balancing
- Reconfigurable scheduling policy
- Multiple resource and policy view
- Simple accounting and economic modeling support (Cluster Bank server)





SCMS: Cluster Management Tool for Beowulf Cluster



- A collection of system management tools for Beowulf cluster
- Package includes
 - Portable real-time monitoring
 - Parallel Unix command
 - Alarm system
 - Large collection of graphical user interface tools for users and system administrator



MPITH



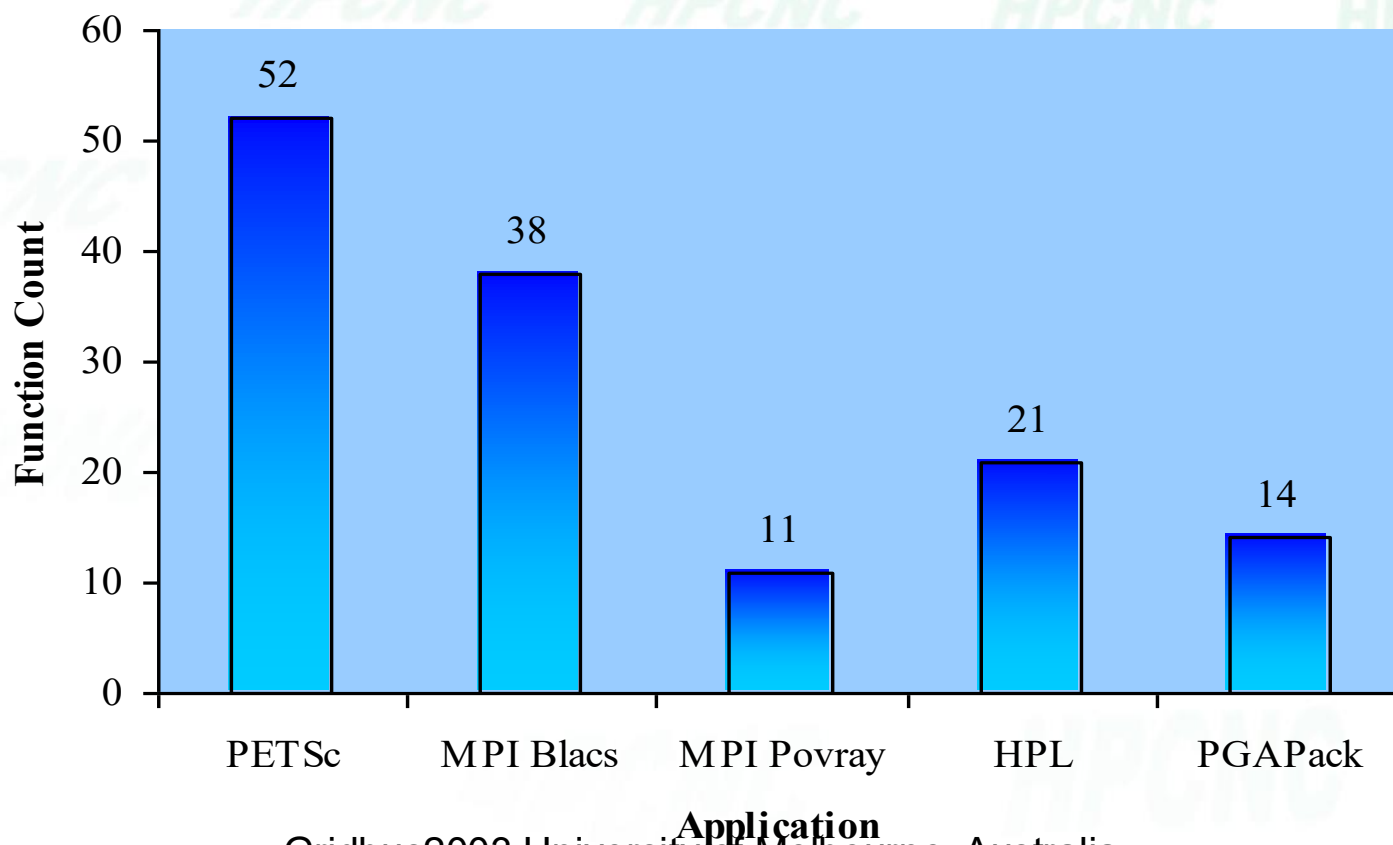
- Small MPI runtime (40-50 functions)
 - OO design
 - C++ Language
 - More than 15000 lines of C++ code
 - Linux operating system
- Architecture
- Selected implementation issue



Preliminaries Study



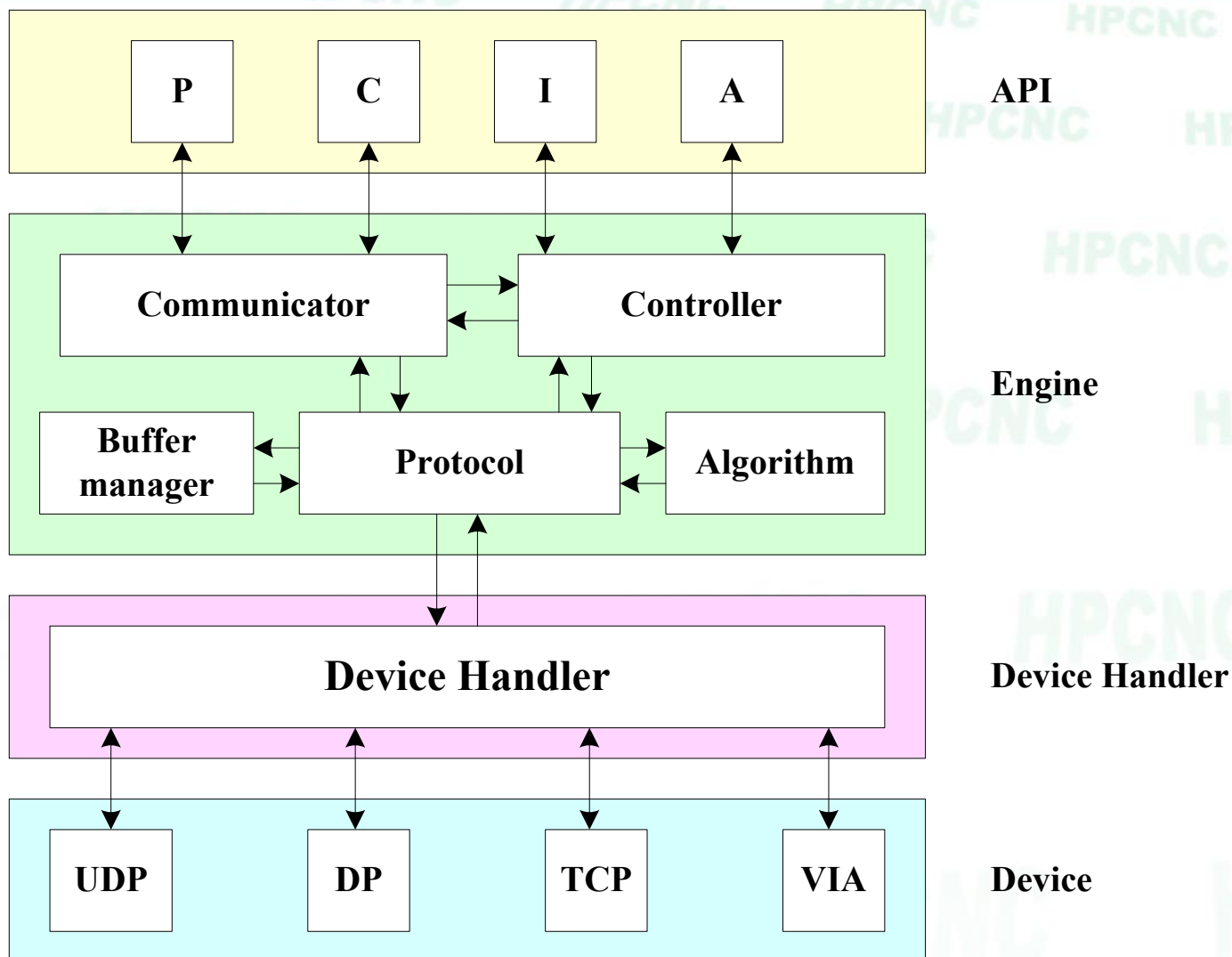
- Only 20-30 functions are used by most developers



Gridbus2003 University of Melbourne, Australia,
June 7, 2003



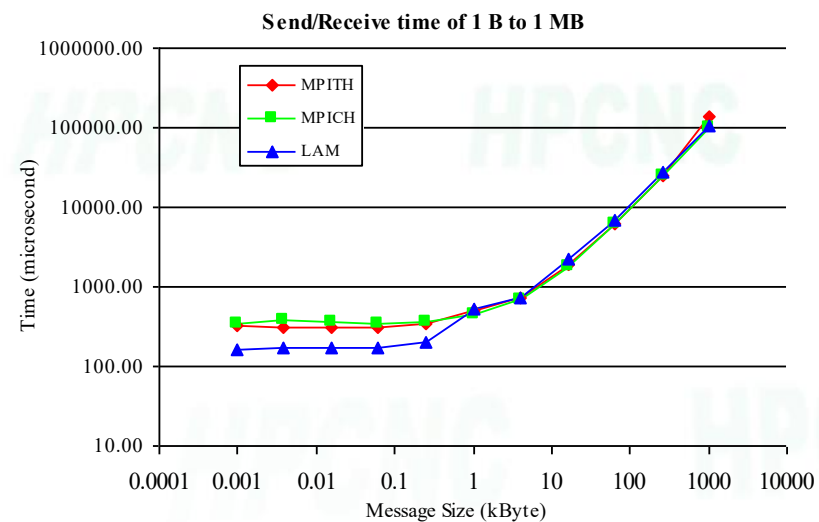
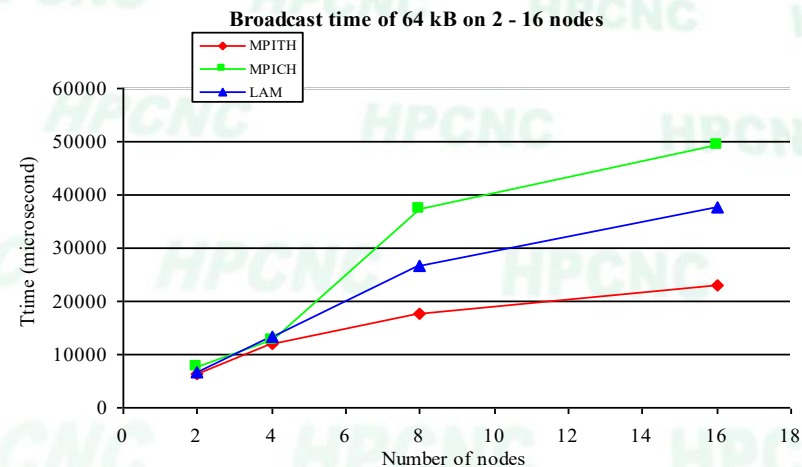
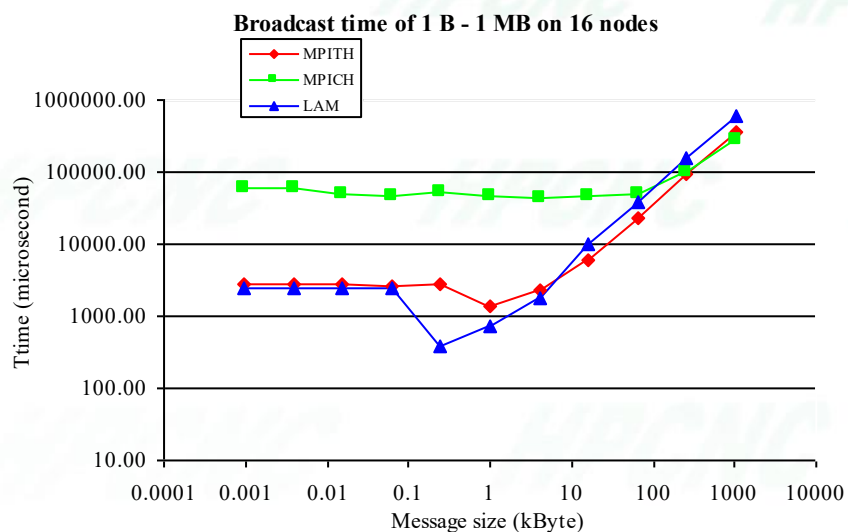
MPITH



Gridbus2003 University of Melbourne, Australia,
June 7, 2003



Broadcast Performance

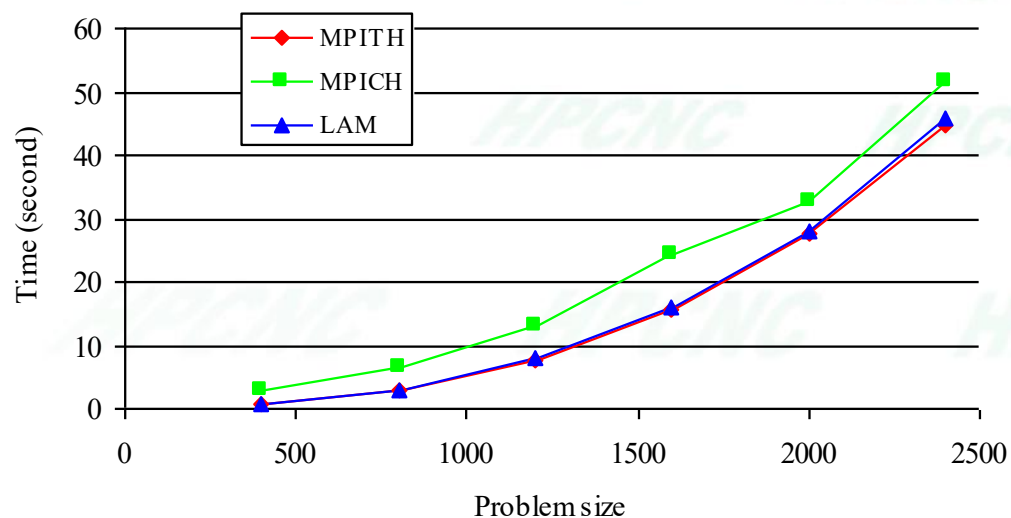




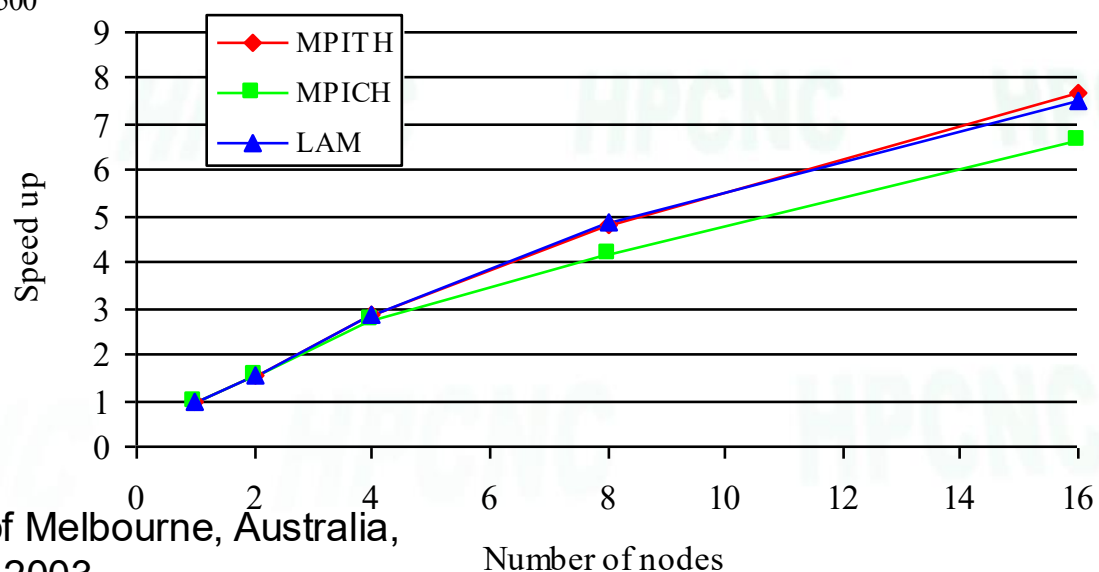
Parallel Gaussian Elimination



Gaussian elimination running time of 400 to 2400 variables on 16 nodes



Speed up of Gaussian elimination running time of 2400 variables on 1 to 16 nodes

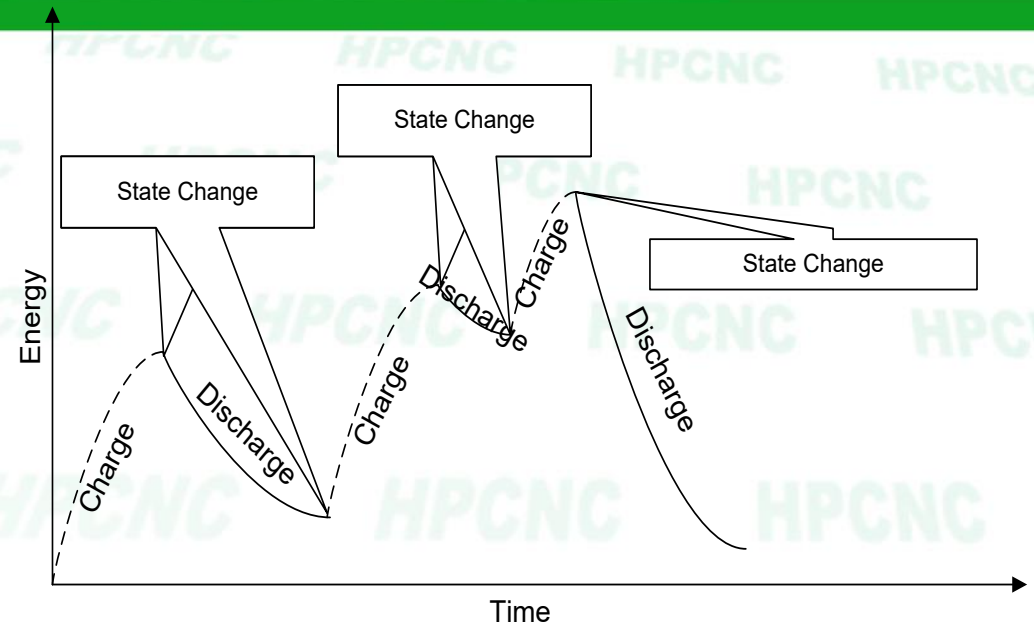




Energy Model for Implicit Coscheduling



- Each process has stored “Energy”
- Process charge/discharge “energy” while it executes
- Charge/Discharge rate is calculated from process statistics
 - Communication Frequency
 - Message Size
 - Amount of running process in the system
- The charging and discharging state changes when communication state changes



- Local scheduling priority are calculated from
 - Static priority
 - Energy level



Implementation Details



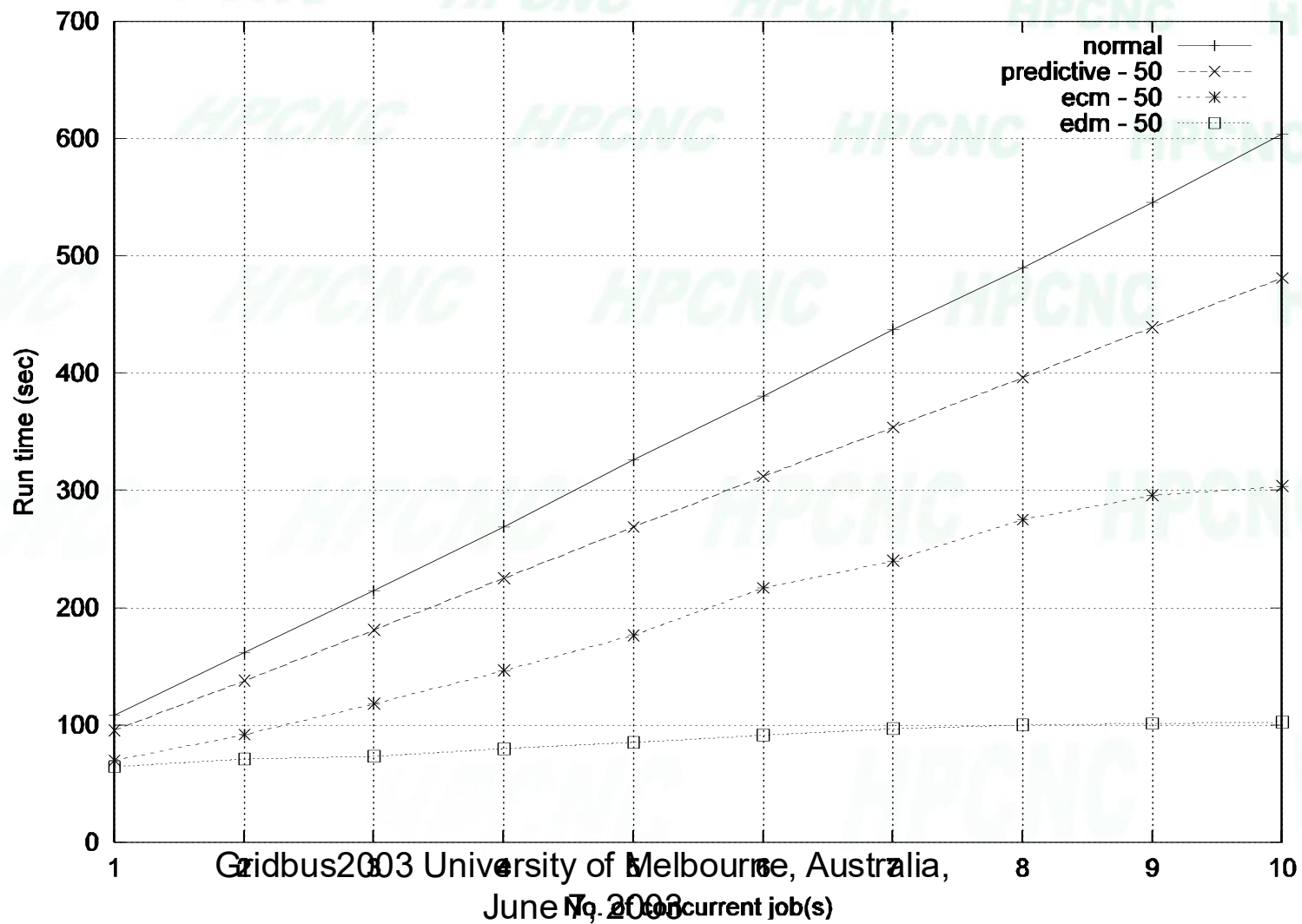
- Implemented in kernel-level as Linux Kernel Module (LKM)
 - kernel version 2.4.19 (the latest at the time)
 - Using Linux timer mechanism to periodically inspect the kernel task queue and adjust the value of each task_struct
 - User need to tell the system which process to do the coscheduling by using command line.
 - _exit system call is trapped to ensure that all internal variable is cleared when process exit



Runtime of parallel application against sequential workload



- Single MG against 1-10 sequential workload

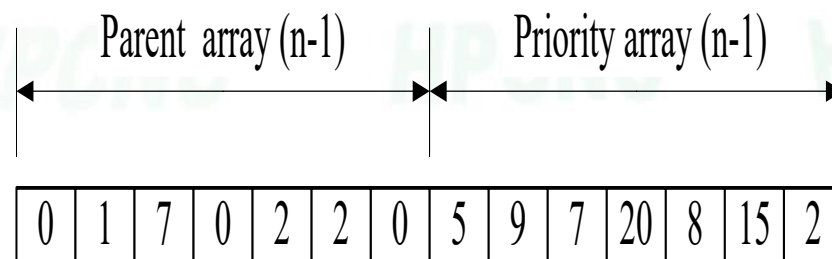
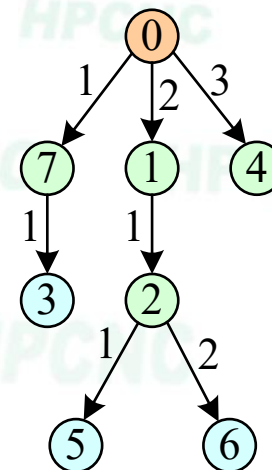




Efficient Collective Communication Algorithm over Grid system



- Genetic Algorithms-based Dynamic Tree (GADT)
 - Heuristic based on genetic algorithm
 - Total transmission time is used as fitness value

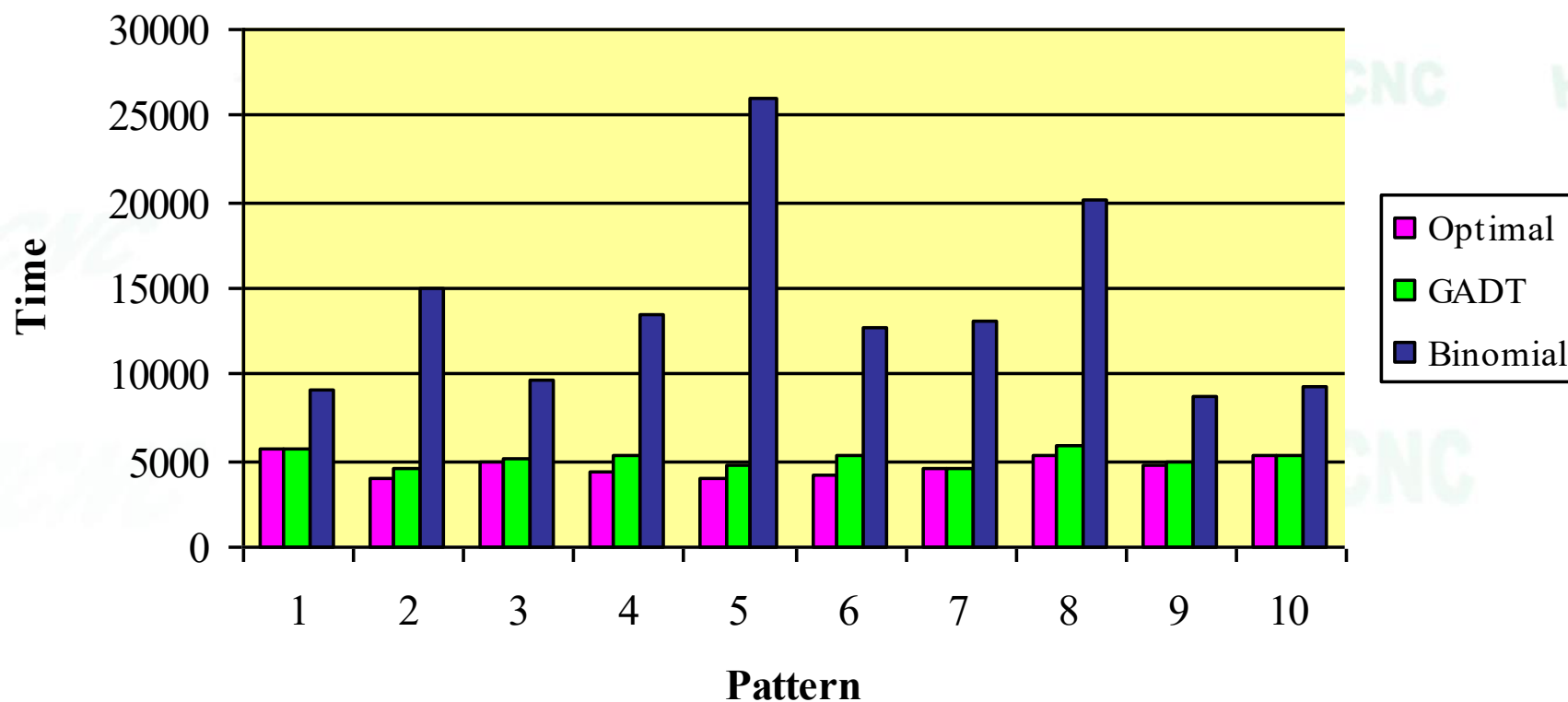




Algorithms Comparison



Transmission of 3 algorithms



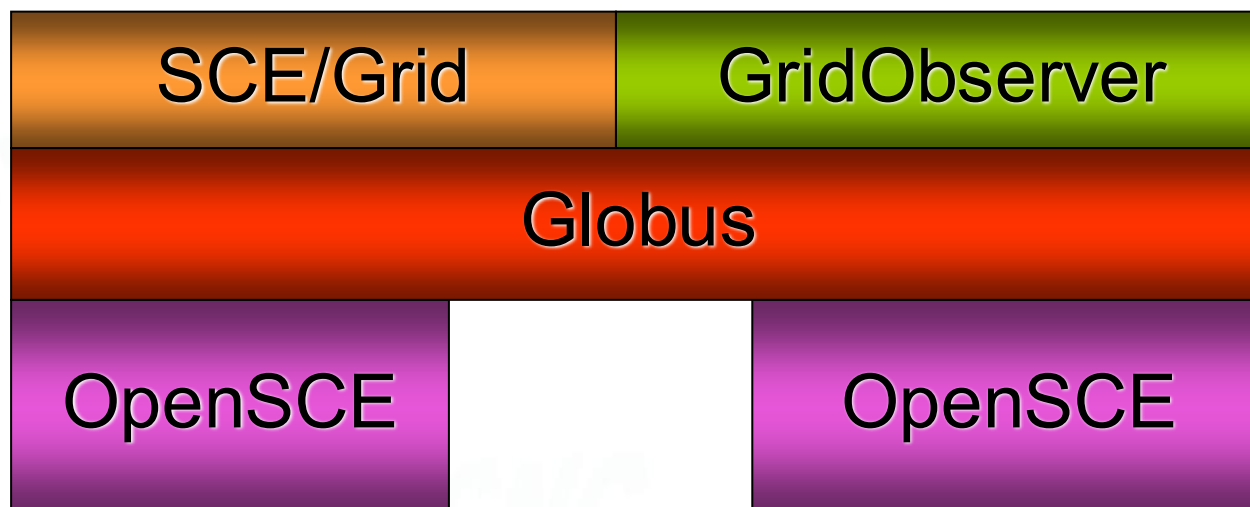
Gridbus2003 University of Melbourne, Australia,
June 7, 2003



OpenSCE and Grid Computing



- Software
 - Grid Observer
 - SCEGrid Grid scheduler
 - HyperGrid Simulator

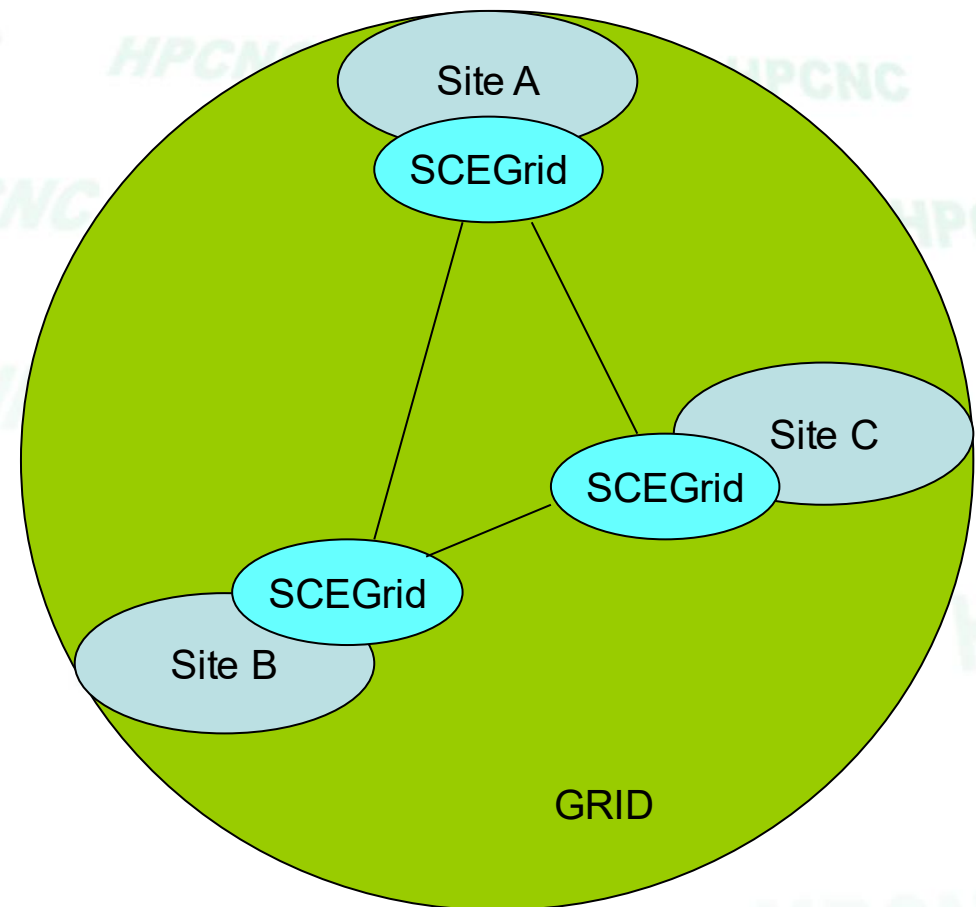




SCE/Grid Architecture

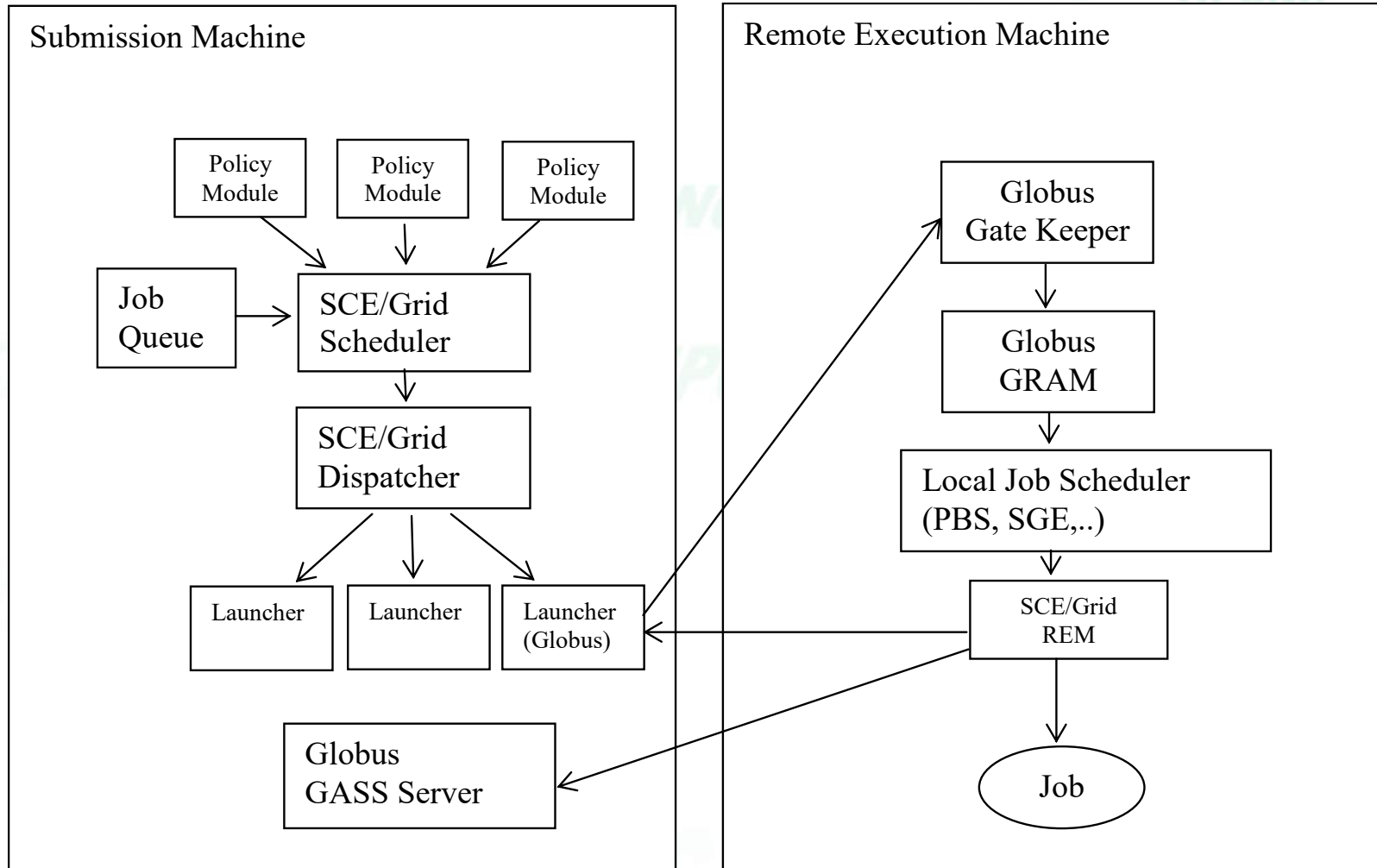


- Distributed resource manager
- Running on top of Globus
- Automatically discovering resources
- Automatically choosing target site





Structure

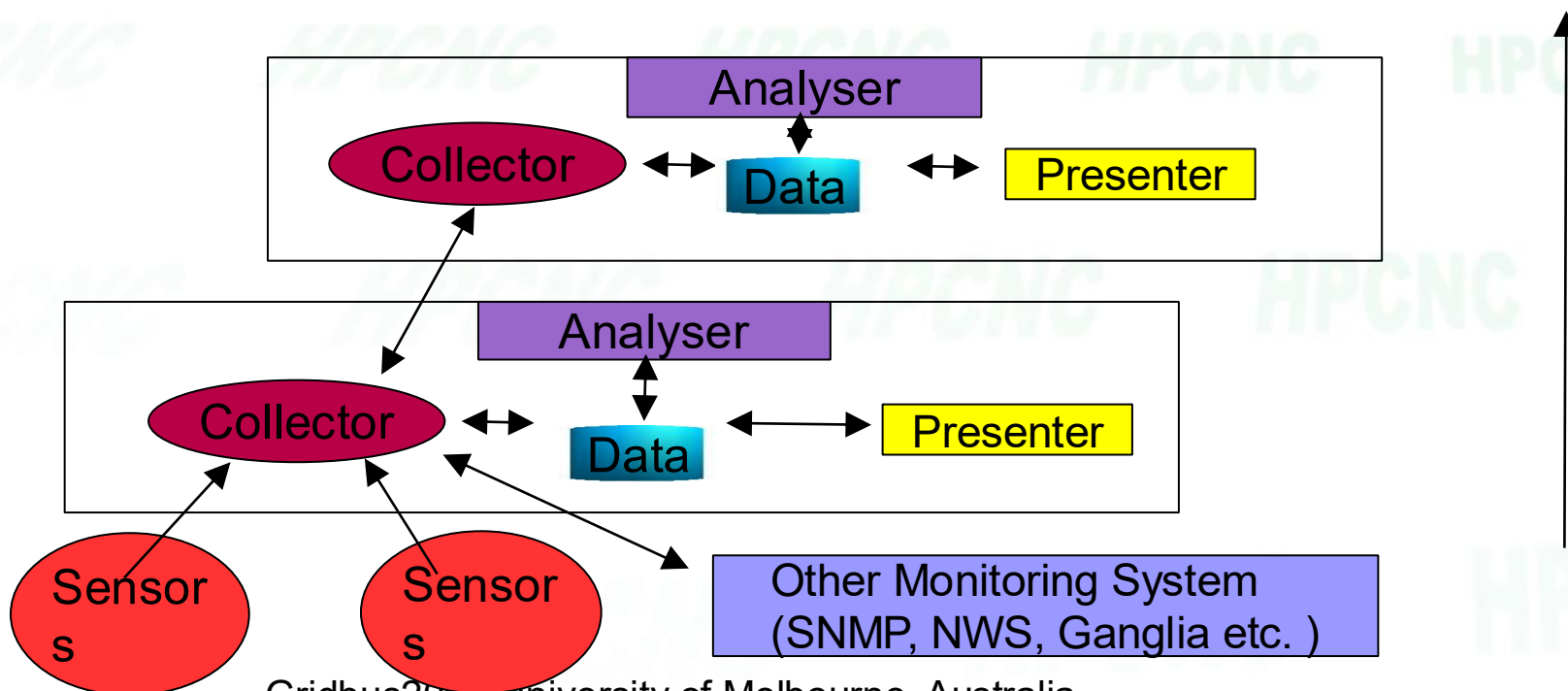




Grid Observer (KU)



- Building technology to monitor the grid
- Software is now used by APGrid Test Bed





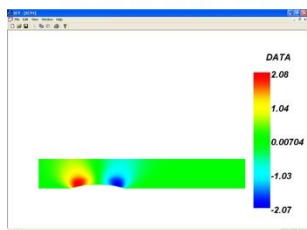
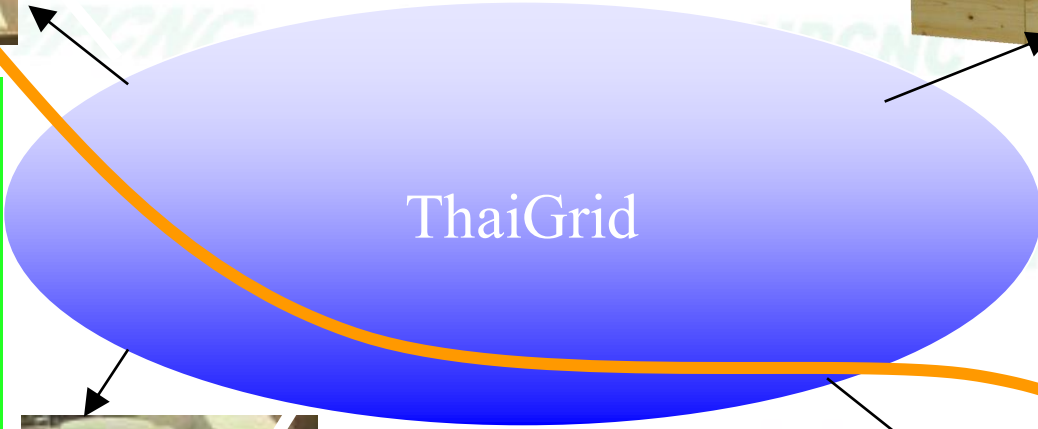
Grid CFD



- Front End
- Sequential Solver
- Visualization

Parallel CFD Solver

Parallel CFD Solver



- Front End
- Sequential Solver
- Visualization





Grid Scheduling



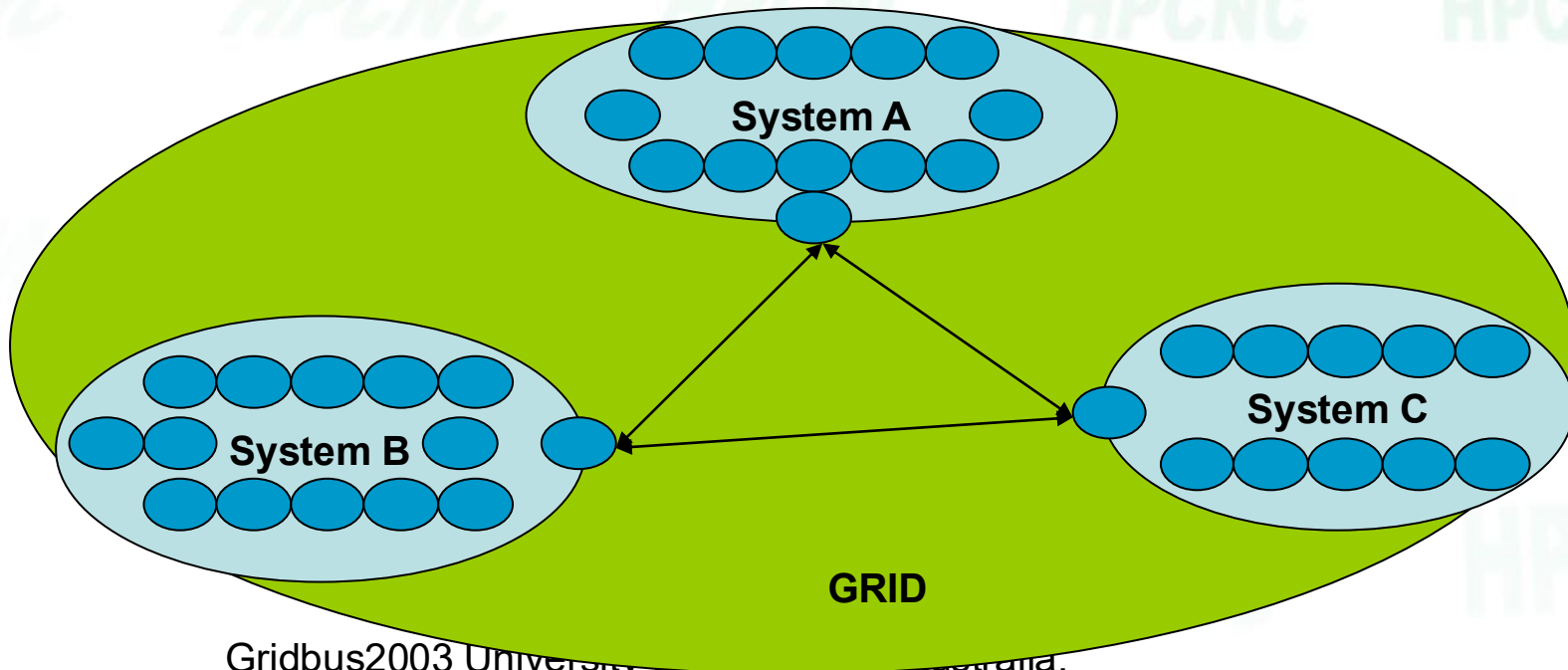
- Problem
 - How to efficiently use distributed/heterogeneous resources
 - Efficiently
 - Cost effectively
- Approach
 - Model the grid scheduling problem
 - Finding good heuristic algorithms
- Grid Scheduling
 - Partial State Scheduling
 - C- sufferage with cost scheduling
 - Vector Space Modeling of computational Grid
 - CFD Task mapping using GA



Grid Model



- Grid
 - Collection of autonomous system
- Autonomous system
 - Collection of *computing node*
 - Contain a *local scheduler*
- Local Scheduler
 - Resource manager
 - Maintain local task queue and manage resource pool e.g. computing node





Grid Vector Space Model



- Each node has **m** resources
- Each system has **n** nodes

$$\vec{N} = [R_1 \quad R_2 \quad R_3 \quad \cdots \quad R_m]$$
$$\vec{S} = \begin{bmatrix} R_{11} & R_{12} & R_{13} & \cdots & R_{1m} \\ R_{21} & R_{22} & R_{23} & \cdots & R_{2m} \\ R_{31} & R_{32} & R_{33} & \cdots & R_{3m} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ R_{n1} & R_{n2} & R_{n3} & \cdots & R_{nm} \end{bmatrix}$$



Execution Model



- Each task has **W** works to be done
- Estimated execution time depends on **execution rate** of each node

$$T = W$$

$$T_{exec\ i} = \frac{W}{\tau_i} \quad \leftarrow \text{execution rate}$$

$$\tau_i = \frac{\gamma_i}{\alpha_i} \quad \begin{array}{l} \leftarrow \text{speed} \\ \leftarrow \text{load} \end{array}$$



Resource Commerce Model (RC)



- Proposed task allocation model on Grid system
 - Batch scheduling
 - Sequential job
 - Economic model : rental cost structure, objective function
 - Framework for several proposed heuristics



RC for On-line scheduling



- Single task
 - On-line
 - Let C_i be rental cost of running the task t on node S_i
 - Result: On-line minimum cost assignment is $O(n \log n)$
- Multiple task
 - Batch
 - Parallel
 - Let C_{ij} be rental cost of running task t_j on node S_i

amount of required resources vector

cost rate vector

$$C_i = [R_1 \quad R_2 \quad R_3 \quad \dots \quad R_m] \begin{bmatrix} R_{t_1} \\ R_{t_2} \\ R_{t_3} \\ \vdots \\ R_{t_m} \end{bmatrix}$$



Objective function for RC model



- p_{ij} = priority index of running job i on machine j
- e_{ij} = execution time of job i on machine j
- Let r_j be ready time of machine j
- Let f_t be time factor
- Let f_{tb} be time balance factor
- Let f_c be cost factor
- Let f_{cb} be cost balance factor

$$p_{ij} = f_t e_{ij} + f_{tb} r_j - f_c C_{ij} - f_{cb} c_j$$



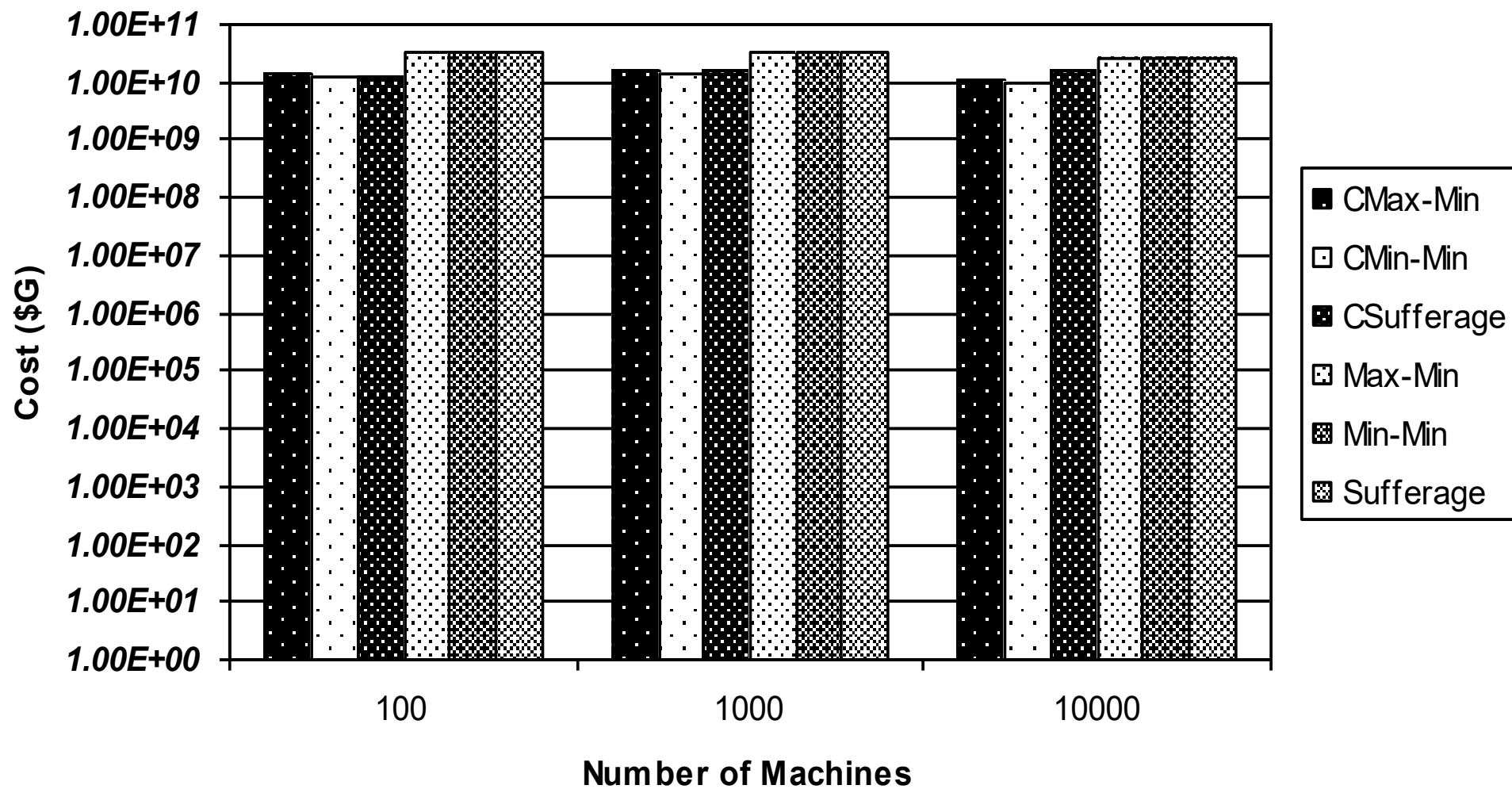
Some Algorithms



- C-Max/Min
- C-Min/Min
- C- Sufferage
- C-Sufferage with Deadline



Cost



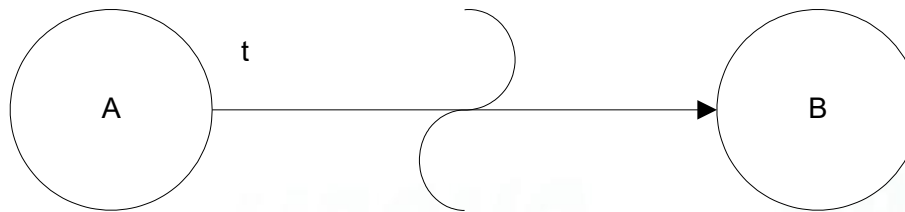
Gridbus2003 University of Melbourne, Australia,
June 7, 2003



Hypersim Simulator



- Discrete event simulation engine from AIT/KU Collaboration
 - C++ Class
 - Event-based Model
 - Fast event processing
- Concept
 - User define the system using event graph
 - When A occurs and condition (i) is true, event B is scheduled to occur at current timeⁱ + t

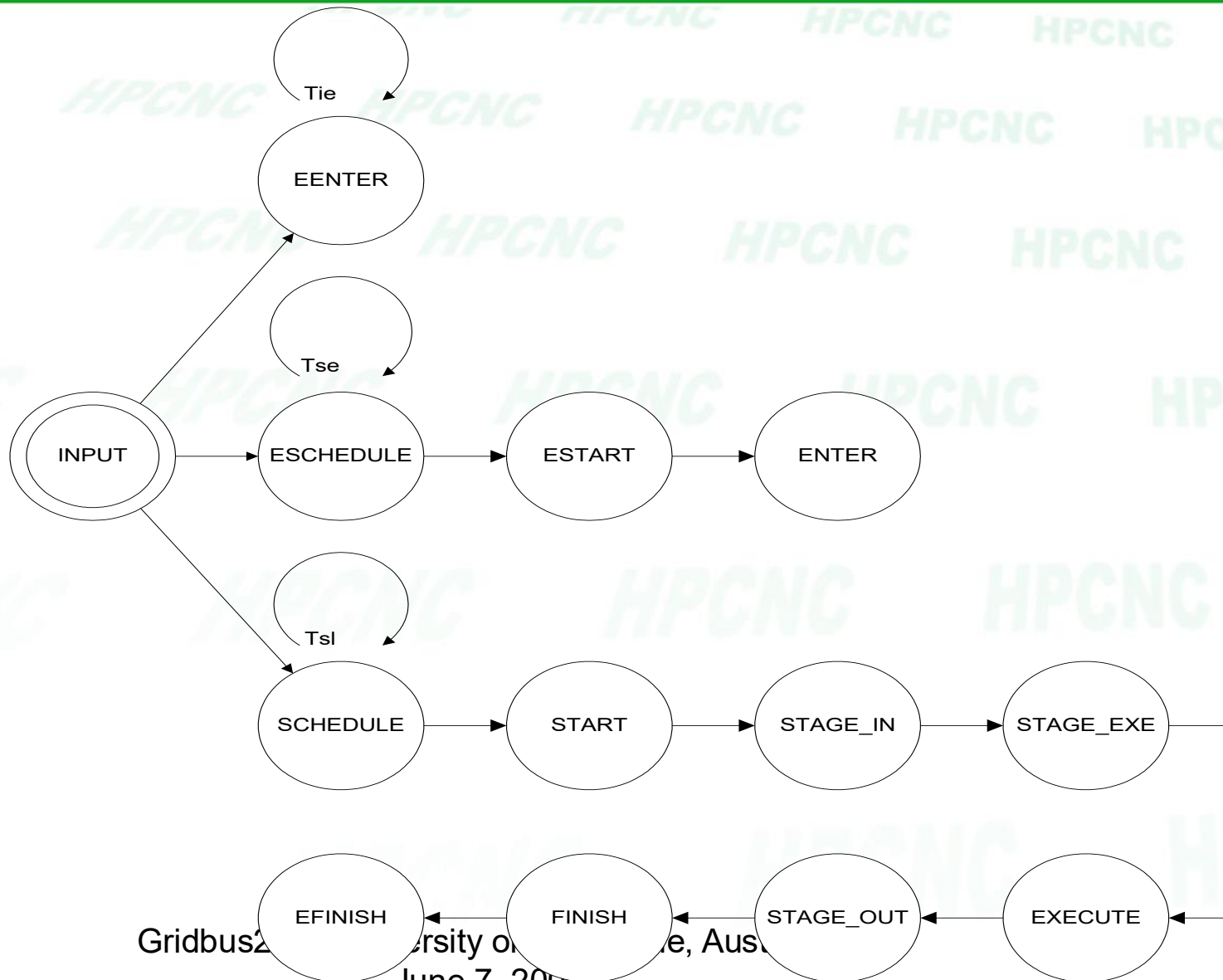


Gridbus2003 University of Melbourne, Australia,

- Hypersim maintain event state, state transition

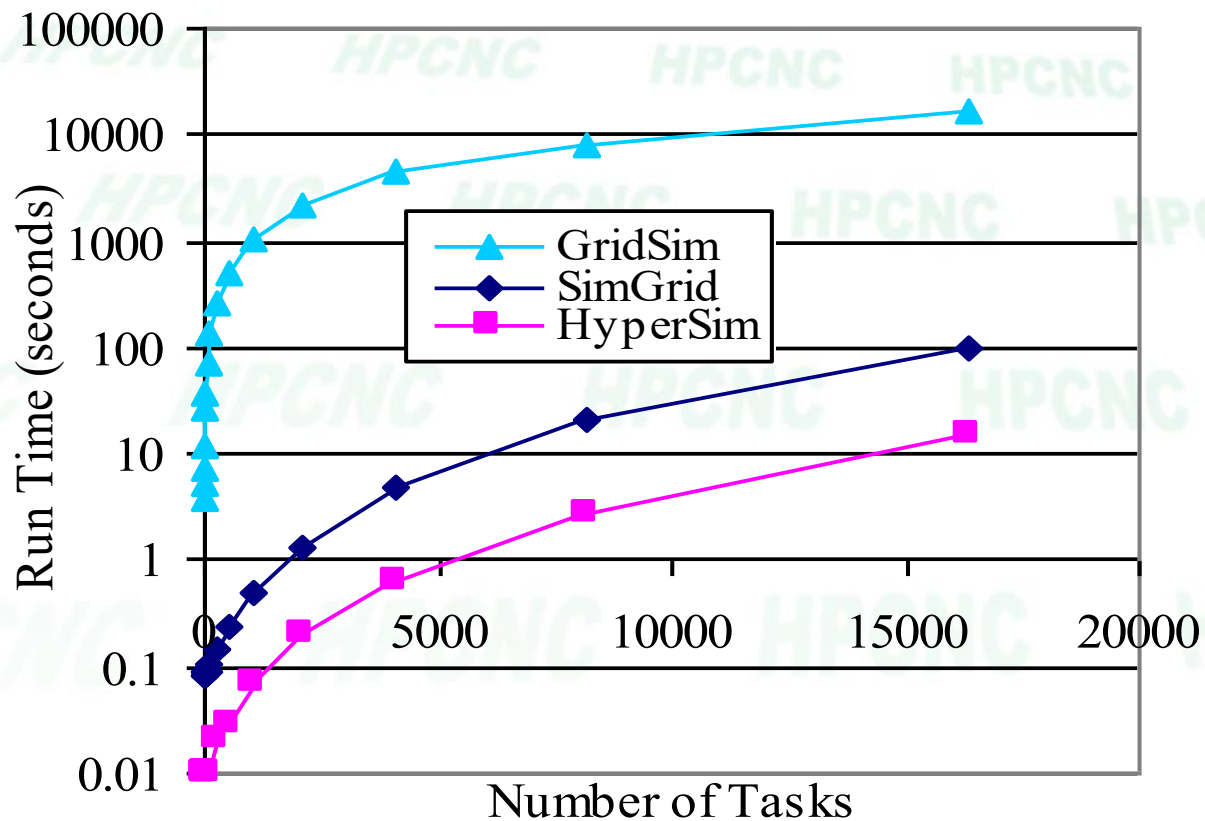
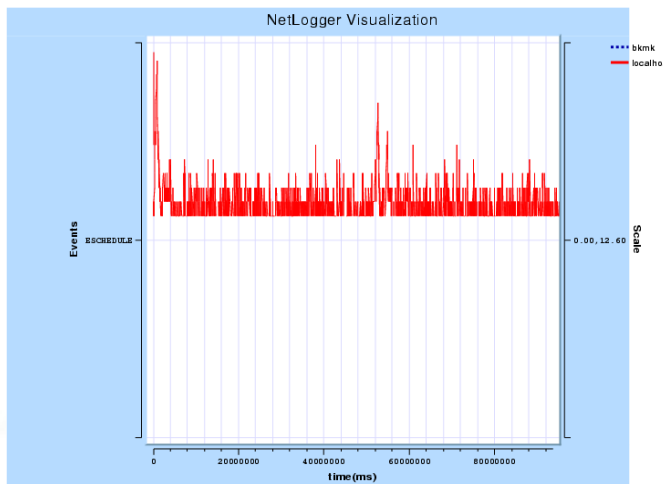


Grid Model





Some Results





Future Work



- More understanding about Grid economy
- Complete our MPI , use it on the grid (before SC2003)
- Many new algorithms
- Tools for ApGrid/ PRAGMA
- Collaboration
 - GridBank Grid Market Interface for OpenSCE scheduler
 - GridScape for our portal



The End



Ha..ha..ha

Gridbus2003 University of Melbourne, Australia,
June 7, 2003



Kasetart University



- Leading multidisciplinary academics institute in Thailand
- Second oldest university in Thailand
- About 25000 students in 5 campuses around the country
- Leading in
 - Biotechnology
 - Computational chemistry
 - Computer science and engineering
 - Agricultural technology

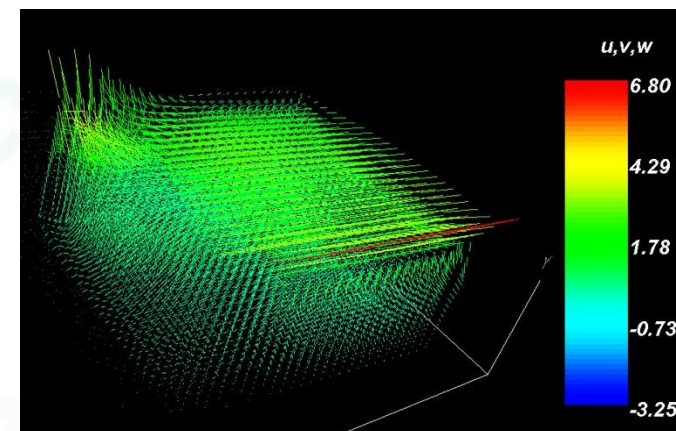
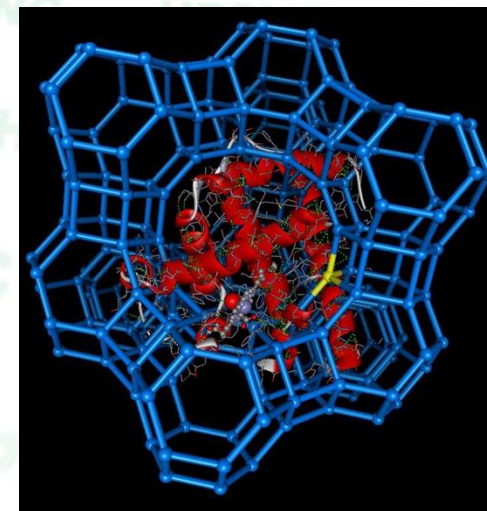




KU HPC Research



- Many advanced research are being pursued by KU researchers
 - Computer-Aided Molecular Modeling and Design of HIV-1 Inhibitors
 - Bioinformatics research to improve rice quality
 - Computational Fluid dynamics for CAD/CAM, vehicle design, clean room
 - VLSI test simulation
 - Massive information and knowledge, analysis, storage, retrieval
- All these research require a massive amount of computing power!

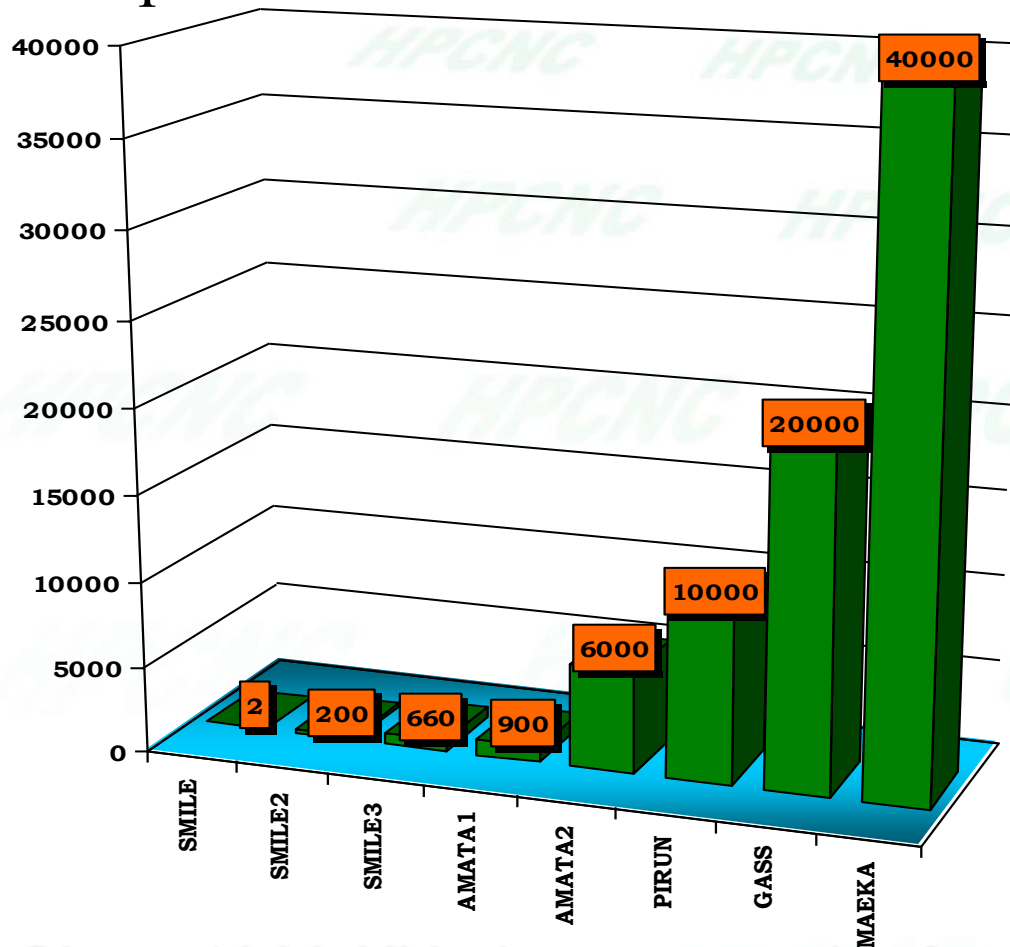




KU Cluster Evolution



Mflops



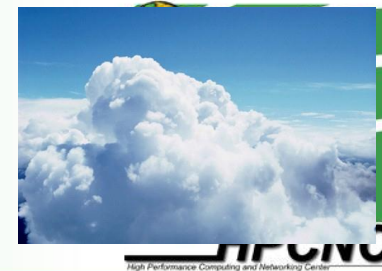
Since 1999 KU always own the fastest
Computing system in Thailand

Gridbus2003 University of Melbourne, Australia,
June 7, 2003



MAEKA System

Massive Adaptable Environment for Kasetart Applications



- Collaboration with AMD Inc.
- Initial Phase
 - 32 processors (16 dual processors node) Opteron system
 - Gigabit Ethernet
 - Massive and scalable storage
 - 50-80 Gigaflaps
- **Fastest computing system in Thailand.**
- Much larger system will be built this year





Structures and Components



User

[1] an user submits a job

Scheduler

LDAP

[2] queries available resources

GIIS/GRIS

GRID

[3] chooses the target site and dispatches the job

Dispatcher

GRAM

[4] submits the job to the target site
[5] waits until finish

Gatekeeper

jobmanager

Local Scheduler
PBS, Condor, SQMS, ...